Davante

19

BASE DE DATOS

Utilización de los objetos

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Declaración de objetos 5

/ 3. Instancias de objetos 6

/ 4. Referencias a atributos de los objetos 7

/ 5. Llamadas a métodos 8

/ 6. Herencia 9

/ 7. Polimorfismo en bases de datos objeto-relacionales 10

/ 8. Método MAP 11

/ 9. Método ORDER 12

/ 10. Palabras clave FINAL, INSTANTIABLE, NO INSTANTIABLE y SELF AS RESULT 13

10.1. FINAL 13

10.2. INSTANTIABLE y NOT INSTANTIABLE 13

10.3. SELF AS RESULT 14

/ 11. Caso práctico 1: "Inicialización de objetos y acceso a sus atributos" 14

ÍNDICE

/ 12. Caso práctico 2: “Práctica con el concepto de herencia en Oracle SQL Express” 15

/ 13. Resumen y resolución del caso práctico de la unidad 16

/ 14. Bibliografía 17

© MEDAC ISBN: 978-84-19885-68-5
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Entender cómo se declaran e inicializan los objetos.

Saber cómo referenciar y manejar los atributos de otros objetos.

Aprender a realizar y gestionar llamadas a métodos.

Entender el uso de la herencia y el polimorfismo en objetos.

Familiarizarse con el manejo de los métodos MAP y ORDER.

Conocer algunas palabras clave de importancia.

/ 1. Introducción y contextualización práctica

En el anterior tema, exploramos las características fundamentales de las bases de datos objeto-relacionales (BDOR) y se presentó el concepto de objeto, junto con sus elementos clave, como atributos y métodos, entre otros. Además, se analizaron varios conceptos de gran importancia en el ámbito de las bases de datos objeto-relacionales, tales como la sobrecarga y el uso de constructores.

Ahora, es fundamental expandir nuestro conocimiento sobre las BDOR y adquirir habilidades en la creación y utilización de objetos en diversos contextos de aplicación, para así comprender mejor las capacidades que estas tecnologías pueden ofrecer. En esta unidad, aprenderemos a declarar e inicializar objetos de distintos tipos, lo que implica asignar valores a sus atributos, y exploraremos cómo acceder a su contenido a través de sus atributos o sus métodos. Además, profundizaremos en temas avanzados como la herencia y el polimorfismo, y en el uso de ciertos métodos y palabras clave esenciales en este entorno.

Abordaremos este conocimiento desde un enfoque completamente práctico, dado que, como hemos señalado anteriormente, la práctica es esencial en estos entornos para lograr un aprendizaje correcto y completo de los conceptos que vamos a explorar.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.



Fig.1. La utilización de los objetos es un proceso básico en las BDOR.

Audio Intro. "Referencias a atributos y llamadas a métodos"

https://on.soundcloud.com/bqAcvoLxWx35vazo6

/ 2. Declaración de objetos

En el tema anterior, abordamos el estudio de la declaración de tipos de datos objeto. Una vez definidos estos tipos, es posible **declarar variables o atributos de ese mismo tipo de dato objeto en otras partes del código**. Estos tipos de datos **pueden ser utilizados como cualquier otro tipo de dato**, **como** por ejemplo **VARCHAR2**, permitiendo que formen parte de tablas, sean parámetros en funciones, entre otras aplicaciones. A continuación, veremos algunos ejemplos para comprender mejor este concepto.

Basándonos en algunos casos analizados en el tema anterior, podríamos **crear variables que almacenen objetos de distintos tipos**, como se muestra en la siguiente imagen:

```
Alumno1 alumno;
Profe1 profesor;
```

Fig.2. Creación de variables para almacenar dos objetos.

Asimismo, podríamos utilizar un **objeto que hayamos creado como parámetro en un procedimiento**, incluyendo aquellos métodos que pertenecen al mismo tipo de objeto. Tomando los ejemplos anteriores, podríamos definir los procedimientos de la siguiente imagen:

```
PROCEDURE incluirAlumno (Alum IN alumno);
PROCEDURE incluirProfesor (Profe IN profesor);
```

Fig.3. Ejemplos de procedimientos.

Estos procedimientos pueden ser llamados pasando como parámetro un objeto del tipo correspondiente, como se puede observar en la siguiente imagen:

```
incluirAlumno (Alumno1)
incluirProfesor (Profe1)
```

Fig.4. Llamada a procedimientos con parámetros.

Además, es posible emplear **objetos como valores de retorno en funciones**, de manera que, al invocar estas funciones, se ejecute la operación deseada y se devuelva un objeto. En la siguiente imagen se muestra un ejemplo de ello:

```
FUNCTION modificarAlumno (a NUMBER) RETURN alumno;
FUNCTION modificarProfesor (a NUMBER) RETURN profesor;
```

Fig.5. Ejemplos de funciones que devuelven objetos.

Es importante destacar que, además de la creación del objeto, es fundamental que este cuente con una serie de valores asociados que lo distingan de otros objetos del mismo tipo. En otras palabras, dentro del tipo de objeto 'alumno' o 'profesor', existirán diversas instancias con características propias que las diferencien de las demás. Estas características están definidas por los atributos del objeto. El proceso mediante el cual **se crea una instancia de un objeto y se asignan valores específicos a sus atributos** se denomina **inicialización del objeto**, y será abordado en el siguiente apartado.

/ 3. Instancias de objetos

Para crear una instancia de un objeto, es decir, para **inicializarlo**, es **necesario utilizar** lo que se conoce como **método constructor**, un concepto que ya se introdujo en el tema anterior. El constructor es un método diseñado específicamente para generar nuevos objetos, es decir, para crear nuevas instancias de un mismo tipo de dato objeto.

La **sintaxis** habitual para **declarar un constructor** sigue este formato, empleando la palabra clave **NEW**, como se muestra en la imagen siguiente:

```
Nombre_objeto := NEW tipo_objeto (atributo1, …, atributoN);
```

Fig.6. Sintaxis para instanciar un objeto.

En la siguiente imagen, se muestra un ejemplo de **creación de una instancia del tipo de dato objeto** 'profesor':

```
profesor1 := NEW profesor ('Antonio', 'Jiménez Sánchez',
                '22222222F','Matemáticas','Avda Lepanto 5');
```

Fig.7. Inicialización de un objeto.

Con este código, hemos **inicializado una instancia del objeto**, asignando valores específicos a cada uno de los atributos definidos en la declaración del tipo de dato objeto.

El método constructor puede ser invocado desde diversas partes del código, de manera similar a como se haría con una función o procedimiento estándar. Además, al llamar a un método constructor, es necesario pasarle ciertos valores como parámetros, que corresponden a los atributos del objeto, tal como vimos anteriormente.

En el caso de Oracle, es **importante que la llamada al método constructor siga el mismo orden en el que los atributos fueron declarados en el tipo de dato objeto**; de lo contrario, el sistema generará un error.

Ilustremos esto con un ejemplo: Supongamos que tenemos un tipo de objeto denominado 'venta', declarado con tres atributos en el siguiente orden: id_venta (INTEGER), Comercial (VARCHAR2), Importe (NUMBER). Sería incorrecto inicializarlo de la manera indicada en la siguiente imagen:

```
venta1 := NEW venta ( 'Ana García',55,1000);
```

Fig.8. Inicialización de un objeto de manera incorrecta.

Lo correcto sería inicializarlo de la forma indicada en esta otra imagen:

```
venta1 := NEW venta (55, 'Ana García',1000);
```

Fig.9. Inicialización de un objeto de manera correcta.

/ 4. Referencias a atributos de los objetos

En este apartado, veremos cómo acceder a los atributos de un objeto y cómo modificarlos y utilizarlos, incluyendo los de objetos anidados.

A. Acceso a atributos de una instancia

Se puede acceder a los atributos de una instancia de un objeto para consultarlos, modificarlos o utilizarlos, haciendo referencia al nombre del objeto seguido del nombre del atributo, separados por un punto. La sintaxis sería la de la siguiente imagen:

```
Nombre_del_objeto.Nombre_del_atributo
```

Fig.10. Sintaxis de acceso al atributo del objeto.

Por ejemplo, para acceder al atributo 'nombre' del objeto 'profesor', la sintaxis sería la de esta otra imagen:

```
profesor1.nombre
```

Fig.11. Acceso al atributo 'nombre' del objeto 'profesor1'.

B. Asignación a una variable

Al acceder a un atributo de un objeto, en realidad estamos obteniendo un valor. Por lo tanto, dicho valor puede ser asignado directamente a una variable, como se ve en la siguiente imagen:

```
Nombre_profe := profesor1.nombre
```

Fig.12. Asignación del valor del atributo 'nombre' del objeto 'profesor1' a la variable 'Nombre_profe'.

C. Uso como parámetro en funciones o procedimientos

También es posible utilizar el valor de un atributo como parámetro en una función o procedimiento, de la misma manera que se haría con una variable o un valor. Se muestra en la siguiente imagen:

```
funcion_de_prueba(profesor1.nombre);
```

Fig.13. Ejemplo de paso por parámetro de una función.

D. Modificación o asignación del valor de un atributo

Además, se puede cambiar o asignar el valor a un atributo directamente, siguiendo la sintaxis de esta imagen:

```
profesor1.nombre := 'Ramiro';
```

Fig.14. Modificación/asignación del valor a un atributo.

E. Acceso a atributos de objetos anidados

Se puede acceder a los atributos de objetos anidados dentro de otros objetos. Si un atributo de un objeto es otro objeto, se puede acceder a sus atributos de manera similar. Por ejemplo, si el objeto 'profesor' es un atributo del objeto 'persona', podemos acceder al nombre del profesor con la sintaxis de la imagen siguiente:

```
persona.profesor1.nombre
```

Fig.15. Acceso a los atributos de un objeto anidado.

Audio 1 "Acceso a atributos no inicializados"

https://on.soundcloud.com/3Hh8ijT79ypoJnoh8

/ 5. Llamadas a métodos

Hasta ahora, hemos visto cómo declarar los métodos dentro de un tipo de dato objeto, pero ¿cómo se utilizan? Vamos a verlo.

> **Recuerda...**
>
> Recuerda que las bases de datos objeto-relacionales permiten encapsular datos y comportamiento en un solo tipo de objeto, facilitando la gestión y alineando el modelo de datos con el del mundo real.

A. Sintaxis

Se pueden invocar los métodos declarados en un tipo de objeto de manera similar a como se llaman funciones o procedimientos. La sintaxis **para invocar un método de un objeto** sigue un esquema parecido al utilizado para acceder a sus atributos: se utiliza el **nombre del objeto seguido del nombre del método, separados por un punto**. En la siguiente imagen podemos verlo:

```
profesor1.getSalario();
```

Fig.16. Llamada al método getSalario de la instancia profesor1.

B. Métodos con parámetros

Si un método requiere recibir **parámetros**, estos se pasan **separados por comas** dentro de los **paréntesis**, como se ve en la siguiente imagen:

```
profesor1.actualizaValor('Ana', 'López', '111111F','Inglés');
```

Fig.17. Llamada a método con parámetros.

C. Acceso a métodos de objetos anidados

Al igual que con los atributos, también es posible **acceder a métodos en objetos anidados**, es decir, en situaciones donde un **objeto es un atributo de otro**. Es importante recordar que el **primer método** debe **devolver** un **objeto del mismo tipo** que el siguiente **método a la derecha**.

```
asignatura.getProfesor().setSalario('1000');
```

Fig.18. Acceso a los métodos de un objeto anidado.

En el ejemplo de la imagen anterior, el método getProfesor debe estar definido en el tipo de objeto 'asignatura' y debe devolver un objeto de tipo 'Profesor', sobre el cual se puede invocar el método setSalario. Es importante seguir esta regla, ya que los métodos se ejecutan de izquierda a derecha.

D. Métodos MEMBER y STATIC

Estos se invocan de manera diferente. Los métodos **MEMBER se llaman sobre instancias específicas de un objeto** y se suelen utilizar para modificar o consultar los valores de sus atributos. En cambio, los métodos **STATIC se invocan directamente sobre el tipo de objeto en sí** y suelen estar destinados a operaciones que afectan a un ámbito más global. En la siguiente imagen, se muestra un ejemplo con MEMBER y STATIC.

```
profesor1.metodo_ejemplo_MEMBER();
profesor.metodo_ejemplo_STATIC();
```

Fig.19. Ejemplo de invocación a un método MEMBER y otro STATIC.

A continuación podrás ver un recurso sobre el punto anterior:

Vídeo 1 "Métodos MEMBER y STATIC"
https://vimeo.com/500402611/62ae12417d

/ 6. Herencia

Es un mecanismo clave en las bases de datos objeto-relacionales que **permite a un tipo de objeto** (**subtipo**) extender a **otro tipo de objeto** (**supertipo**) y **heredar sus atributos y métodos**. Esto facilita la **reutilización de estructuras de datos** y la **creación de jerarquías** que reflejan relaciones naturales en los datos.

En las bases de datos objeto-relacionales, la herencia **facilita la generalización y especialización de tipos**, permitiendo que un subtipo no solo **herede** las **características del supertipo**, sino que también **defina atributos y métodos** adicionales que son **específicos** de su caso particular.

Para implementar la **herencia en Oracle**, se utiliza el **comando UNDER al declarar un subtipo**, lo que indica que este hereda de un supertipo previamente definido.

Además, Oracle permite **controlar la extensión de tipos** mediante las siguientes palabras clave:

- **NOT FINAL**: Permite que un tipo de dato objeto pueda ser utilizado como base para otros subtipos.
- **FINAL**: Impide que otros tipos de objeto puedan heredar del objeto en cuestión, estableciendo así el límite de la jerarquía.

En cualquier implementación de herencia, es **fundamental** comprender que, **al crear una instancia de un subtipo**, **es obligatorio proporcionar valores para todos los atributos definidos en su jerarquía**, **incluidos los heredados del supertipo**. Esto asegura que el objeto resultante esté completamente definido y operativo, respetando la integridad de los datos a través de la jerarquía de tipos.

En la siguiente imagen se muestra un ejemplo de la creación de un objeto 'estudiante', que hereda del tipo de objeto 'persona':

```
CREATE TYPE  persona AS OBJECT (
    DNI VARCHAR2(9),
    nombre VARCHAR2 (20),
    apellidos VARCHAR2 (50)
) NOT FINAL;

CREATE TYPE  estudiante UNDER persona(
    Num_matricula INTEGER(6),
    Ciclo VARCHAR(50)
) FINAL;

DECLARE
    Alumno1 estudiante;
BEGIN
    Alumno1 := NEW estudiante ('111111111B',
                'Ana', 'García', '012321', 'DAW');
END;
```

Fig.20. Ejemplo de herencia.

Enlaces de interés...

En el siguiente enlace, encontrarás un vídeo en el que podrás descubrir algunos ejemplos de definición de tipos de dato objeto con herencia en SQL Developer. Podrás ampliar tus conocimientos sobre estos: https://www.youtube.com/watch?v=ixebBEnZuq0

/ 7. Polimorfismo en bases de datos objeto-relacionales

El **polimorfismo** es una característica que **permite que diferentes tipos de objetos respondan a la misma llamada de método de manera diferente**. En bases de datos objeto-relacionales, se implementa mediante la definición de métodos en un supertipo y la posibilidad de sobrescribir esos métodos en los subtipos.

En la siguiente imagen se muestra un ejemplo de la creación de un supertipo en el que se implementa un método:

```
CREATE TYPE persona AS OBJECT (
    DNI VARCHAR2(9),
    nombre VARCHAR2(20),
    apellidos VARCHAR2(50),
    MEMBER FUNCTION obtenerIdentificacion
            RETURN VARCHAR2
) NOT FINAL;

CREATE TYPE BODY persona AS
    MEMBER FUNCTION obtenerIdentificacion
            RETURN VARCHAR2 IS
    BEGIN
        RETURN DNI;
    END;
END;
```

Fig.21. Creación de supertipo.

> **¡! Sabías que...**
>
> La herencia y el polimorfismo en BDOR se inspiraron en la programación orientada a objetos de los 80 y 90. Postgres, desarrollado en la Universidad de California, Berkeley, fue pionero en implementarla, permitiendo manejar jerarquías de tipos y lógica en la base de datos.

Una vez creado el supertipo, podemos crear un subtipo que lo extienda, es decir, que herede del supertipo, y pueda sobreescribir un método de este, como se muestra en la siguiente imagen:

```
CREATE TYPE estudiante UNDER persona (
    Num_matricula INTEGER,
    Ciclo VARCHAR2(50),
    OVERRIDING MEMBER FUNCTION obtenerIdentificacion
                RETURN VARCHAR2
) FINAL;

CREATE TYPE BODY estudiante AS
    OVERRIDING MEMBER FUNCTION obtenerIdentificacion
                RETURN VARCHAR2 IS
    BEGIN
        RETURN DNI || ' - ' || TO_CHAR(Num_matricula);
    END;
END;
```

Fig.22. Creación de subtipo 'estudiante'.

En este ejemplo, la función obtenerIdentificacion() es definida en el supertipo 'persona' y sobrescrita en el subtipo 'estudiante' para incluir también el número de matrícula. **Para sobreescribir un método del supertipo se utiliza la palabra clave 'OVERRIDING'**.

/ 8. Método MAP

Se utiliza para **convertir un objeto a un valor simple** (como un número o una cadena), que Oracle puede usar **para ordenar y comparar instancias de ese objeto**. Por ejemplo, en un tipo de objeto Profesor, se podría definir un método MAP que devuelva el salario del profesor. Este valor sería utilizado por Oracle para ordenar o comparar dos objetos de tipo Profesor, determinando cuál de ellos tiene un salario mayor.

Un método MAP debe ser **una función sin parámetros y ha de devolver alguno de los siguientes tipos de datos**:

- DATE.
- NUMBER.
- VARCHAR2.
- INTEGER.
- CHARACTER.
- INTEGER.
- REAL.

Solo puede existir un único método de tipo MAP para cada tipo de objeto que se defina. Este método **se declara dentro de la definición** del tipo de objeto, **utilizando la palabra clave MAP** para identificar la función correspondiente. En la siguiente imagen, se muestra un ejemplo aplicado al tipo de objeto Profesor:

```
CREATE OR REPLACE TYPE profesor AS OBJECT (
    nombre VARCHAR2(50),
    apellidos VARCHAR2(100),
    dni VARCHAR2(9),
    salario INTEGER(5),
    asignatura VARCHAR2(50)
    MAP MEMBER FUNCTION ordenProfes RETURN VARCHAR2
```

Fig.23. Ejemplo de uso de función MAP.

Es esencial **definir correctamente el valor a devolver en el cuerpo del método**. En nuestro ejemplo, se podría ordenar a los profesores por DNI, nombre, apellidos, salario, etc. Si decidimos devolver el DNI, el método se vería como se muestra en la siguiente imagen:

```
CREATE OR REPLACE TYPE BODY profesor AS
    MAP MEMBER FUNCTION ordenProfes
        RETURN VARCHAR2 IS
    BEGIN
        RETURN (dni);
    END ordenProfes;
END;
```

Fig.24. Cuerpo de método MAP.

/ 9. Método ORDER

Se utiliza para **ordenar objetos del mismo tipo**, **comparándolos mediante los atributos del propio objeto y de otro**, que se pasa como argumento.

Para **designar el método de ordenación** se utiliza la **palabra clave ORDER** antes de su declaración, similar a MAP. Es esencial que **el parámetro recibido sea del mismo tipo que el objeto** para que la comparación sea válida.

El método ORDER **devuelve un valor de tipo INTEGER** para indicar la relación de orden entre los dos objetos comparados:

- **Un valor positivo**, si el **segundo** objeto es **mayor** que el **primero**.
- **Un valor negativo**, si el **primer** objeto es **mayor** que el **segundo**.
- **El valor cero**, si ambos objetos son **iguales**.
- **El valor NULL**, **si cualquiera** de los parámetros proporcionados al método **es NULL**.

Estos resultados permiten definir claramente el orden entre objetos del mismo tipo, facilitando así las operaciones de comparación y ordenación.

Enlaces de interés...

En el siguiente enlace, encontrarás la documentación oficial de Oracle con la que podrás ampliar tus conocimientos sobre los métodos MAP y ORDER que acabamos explicar: https://docs.oracle.com/en/database/oracle/oracle-database/21/lnpls/element-specification.html

De igual manera que en el caso de MAP, **un tipo de objeto únicamente puede contener un método ORDER**. En el ejemplo de la siguiente imagen vamos a definir un método ORDER para ordenar objeto de tipo 'profesor':

```
CREATE OR REPLACE TYPE BODY profesor AS
    ORDER MEMBER FUNCTION ordenProfes (p profesor)
            RETURN INTEGER IS
    BEGIN
        IF (SELF.dni) < (p.dni) THEN RETURN -1;
        ELSIF (SELF.dni) > (p.dni) THEN RETURN 1;
        ELSE RETURN 0;
        END IF;
    END;
END;
```

Fig.25. Cuerpo de método ORDER.

La palabra clave **SELF** se utiliza **para referirse al objeto actual dentro del método**. En un tipo de objeto, **solo se puede declarar un método MAP o ORDER**, **no ambos**. **Para ordenar un gran número de objetos**, **es preferible usar MAP**, ya que suele ser más eficiente.

Vídeo 2. "Métodos MAP y ORDER con Oracle SQL Express"
https://bit.ly/3ieotPG

/ 10. Palabras clave FINAL, INSTANTIABLE, NO INSTANTIABLE y SELF AS RESULT

Además de los métodos y tipos de métodos que hemos visto hasta ahora, existen una serie de palabras clave en Oracle que son esenciales para definir el comportamiento y las restricciones de los tipos de objetos en bases de datos objeto-relacionales.

Estas palabras clave permiten a los desarrolladores controlar cómo se utilizan, instancian y manipulan los objetos en diferentes escenarios, asegurando que la integridad y el diseño de la base de datos se mantengan intactos.

Entre estas palabras clave se encuentran **FINAL**, **INSTANTIABLE**, **NOT INSTANTIABLE**, y **SELF AS RESULT**, cada una con finalidades técnicas muy concretas y específicas.

10.1. FINAL

En el contexto de la herencia, es importante que ciertos métodos mantengan su comportamiento original, sin posibilidad de ser modificados por subtipos. Para lograr esto, Oracle permite que los métodos sean declarados con la palabra clave **FINAL**.

Un método marcado como **FINAL** asegura que su **implementación no pueda ser sobrescrita en ningún subtipo**, preservando así la lógica y la integridad de los procesos que dependen de ese método.

Imaginemos un sistema de gestión de empleados donde el cálculo del salario debe seguir una lógica específica y no ha de ser alterado en ningún caso.

Para lograr esto, se puede declarar el método calcularSalario en el tipo Empleado como **FINAL**, tal y como se muestra en la siguiente imagen:

```
CREATE OR REPLACE TYPE Empleado AS OBJECT (
    id NUMBER,
    nombre VARCHAR2(50),
    salarioBase NUMBER,

    FINAL MEMBER FUNCTION calcularSalario RETURN NUMBER
);
```

Fig.26. Método FINAL.

En este ejemplo, al marcar el método como **FINAL**, garantizamos que cualquier tipo derivado de Empleado, como EmpleadoTI o EmpleadoRH, utilice la misma lógica para calcular el salario, sin posibilidad de alteración.

Este enfoque permite mantener la coherencia en el cálculo de salarios en toda la organización, independientemente del tipo de empleado.

10.2. INSTANTIABLE y NOT INSTANTIABLE

Igual que en lenguajes de programación como Java, en Oracle es posible definir tipos de dato objeto que sean abstractos.

Un tipo de dato abstracto no puede ser instanciado, es decir, únicamente sirve como supertipo o para agrupar métodos estáticos que puedan ser utilizados por otros elementos de la base de datos.

Para indicar esto, deben emplearse las siguientes palabras clave:

- **NOT INSTANTIABLE**: Indica que un tipo de objeto **no puede tener instancias**.
- **INSTANTIABLE**: Señala que un tipo de objeto **sí puede tener instancias**. Cuando un **subtipo hereda** de un **supertipo no instanciable**, si queremos que **pueda tener instancias**, se debe **indicar obligatoriamente** la palabra clave **INSTANTIABLE**.

En la siguiente imagen se muestra un ejemplo de cómo definir un tipo de objeto como no instanciable o abstracto.

```
CREATE TYPE persona AS OBJECT (
    DNI VARCHAR2(9),
    nombre VARCHAR2 (20)
) NOT INSTANTIABLE ;
```

Fig.27. Ejemplo de NOT INSTANTIABLE.

10.3. SELF AS RESULT

Las palabras clave '**SELF AS RESULT**' son esenciales en Oracle cuando se trata con constructores personalizados en tipos de objetos. En Oracle, cuando se define un constructor personalizado, es obligatorio utilizar **SELF AS RESULT** para indicar que el método constructor devolverá una instancia del objeto recién creado.

Es especialmente útil cuando se requieren validaciones o asignaciones de valores por defecto en el proceso de creación del objeto.

En la siguiente imagen se muestra un ejemplo de cómo utilizar **SELF AS RESULT** en un constructor personalizado para el tipo de objeto Profesor.

```
CONSTRUCTOR FUNCTION profesor(nombre VARCHAR2,
                              apellido VARCHAR2)
        RETURN SELF AS RESULT
```

Fig.28. Ejemplo de SELF AS RESULT.

Este constructor inicializa los atributos nombre y apellido del objeto Profesor. Al utilizar **SELF AS RESULT**, Oracle garantiza que el objeto Profesor recién creado sea devuelto correctamente y esté listo para su uso inmediato.

/ 11. Caso práctico 1: "Inicialización de objetos y acceso a sus atributos"

Planteamiento: Vamos a poner en práctica la inicialización de objetos y el acceso a sus atributos, aplicando los conceptos que hemos estudiado previamente.

Nudo: Emplearemos los tipos de objeto definidos en el caso práctico n.º 1 del tema anterior: dirección, persona y prueba.

Primero, procederemos a crear una instancia para cada uno de estos objetos, inicializando sus respectivos atributos. Luego, accederemos al atributo 'calle' del objeto 'dirección' y modificaremos el atributo 'apellidos' en la instancia del objeto 'persona'. Por último, seleccionaremos un atributo del objeto 'prueba' y realizaremos un cambio en su valor.

Desenlace: En las siguientes imágenes se muestran las instrucciones necesarias para crear las instancias y modificar los atributos:

```
Direccion1 := NEW direccion ('Avda Fuente de los Álamos',
                             'Pontevedra',36002);
Persona1 := NEW persona ('María del Carmen', 'Castro Gil',
                         'Calle Pazos 7', 14/06/2023);
```

Fig.29. Creación de instancias de dirección y persona.

A continuación, accedemos al atributo 'calle' del objeto 'dirección':

```
Direccion1.calle
```

Fig.30. Acceso al atributo 'calle'.

Y modificamos el valor del atributo 'apellidos' de la instancia creada para el tipo de objeto 'persona':

```
Persona1.apellidos := 'Cerviño Loira';
```

Fig.31. Asignación de valor a atributo.

/ 12. Caso práctico 2: "Práctica con el concepto de herencia en Oracle SQL Express"

Planteamiento: En esta ocasión, aplicaremos la teoría que hemos aprendido en los apartados anteriores, enfocándonos específicamente en el caso de la herencia.

Nudo: Con este propósito, llevaremos a cabo, en Oracle SQL Express, el ejemplo de herencia empleado en el tema, creando un tipo de objeto 'persona' y otro tipo de objeto 'estudiante'.

Desenlace: En la siguiente imagen se muestra el código con el que definimos el objeto PERSONA, especificando NOT FINAL, dado que el objeto ESTUDIANTE derivará de él:

```
CREATE TYPE PERSONA1 AS OBJECT(
    DNI VARCHAR2(9),
    nombre VARCHAR2(20),
    apellidos VARCHAR2(50)
)NOT FINAL;
```

Fig.32. Definición del tipo NOT FINAL Persona.

Una vez creado, procedemos a crear el objeto ESTUDIANTE, para que herede del objeto PERSONA creado anteriormente. En la siguiente imagen puede verse el código necesario:

```
CREATE TYPE ESTUDIANTE UNDER PERSONA1(
    matricula INTEGER(6),
    ciclo VARCHAR2(50)
)FINAL;
```

Fig.33. Definición del tipo FINAL ESTUDIANTE que extiende a PERSONA.

Por último, creamos un objeto 'estudiante' utilizando los atributos de PERSONA y de ESTUDIANTE. En la siguiente imagen se muestra el código necesario:

```
DECLARE
  alumno1 ESTUDIANTE;
  BEGIN
      alumno1 := NEW ESTUDIANTE('111111111','Ana','García','123456','DAW');
  END;
```

Fig.34. Creación del objeto 'persona' con los atributos.

/ 13. Resumen y resolución del caso práctico de la unidad

Este tema ha profundizado en el uso de objetos en bases de datos objeto-relacionales. Comenzamos con la declaración e inicialización de objetos, y continuamos con el acceso a atributos y la invocación de métodos.

Se exploraron la herencia y el polimorfismo, permitiendo a los subtipos sobrescribir métodos heredados. También se estudiaron los métodos **MAP** y **ORDER** para ordenar y comparar objetos.

Finalmente, se introdujeron las palabras clave **FINAL**, **INSTANTIABLE**, **NOT INSTANTIABLE** y **SELF AS RESULT**, que controlan la herencia, la instanciación y la correcta inicialización de los objetos.

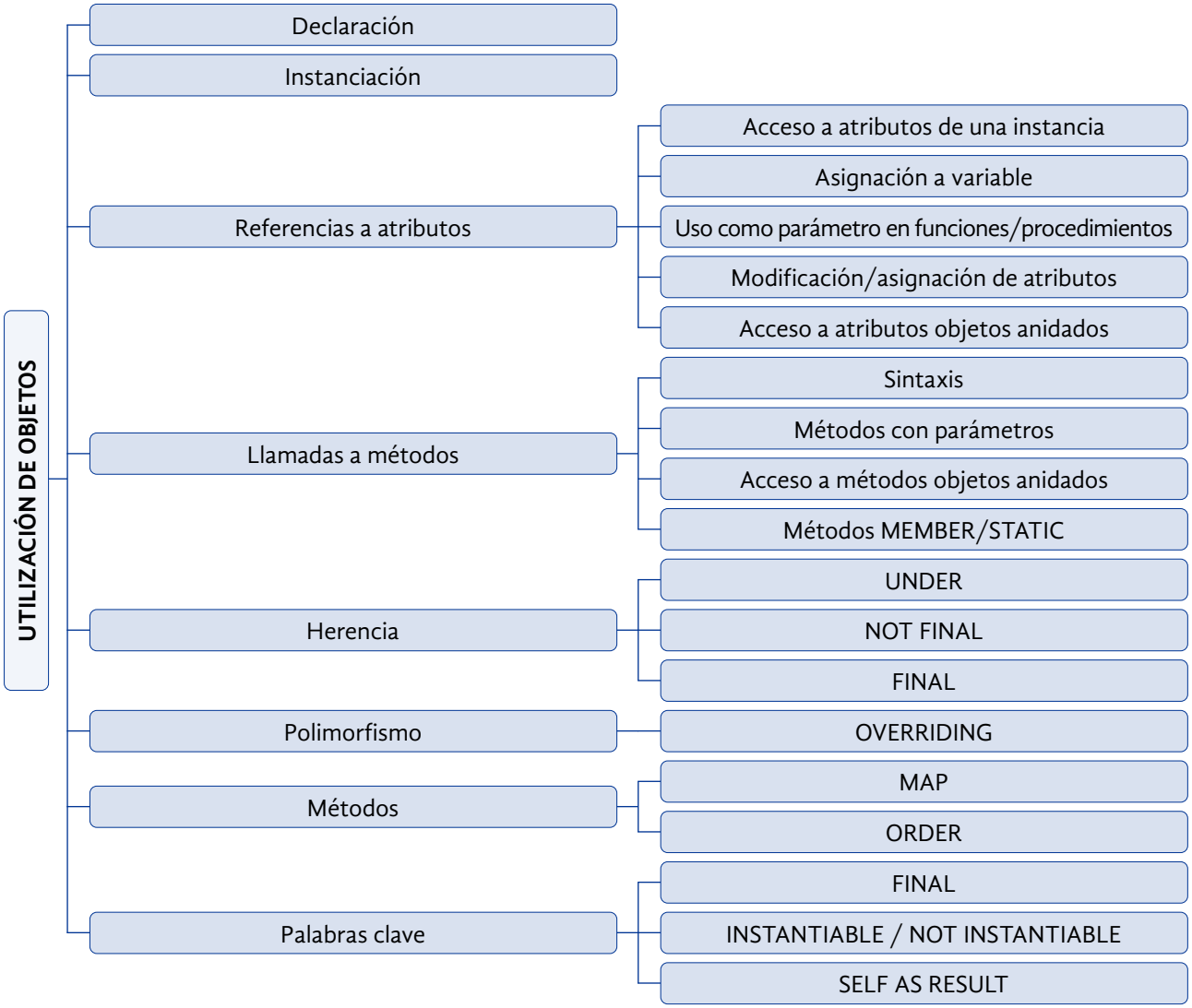


Fig.35. Esquema resumen del tema .

Resolución del caso práctico de la unidad

Como hemos visto a lo largo de este tema, y de modo similar a lo que sucede en cualquier lenguaje de programación, como PL/SQL, el acceso a los atributos y métodos de los objetos es un elemento clave y esencial para el diseño y funcionamiento de las bases de datos objeto-relacionales.

Las opciones y funcionalidades disponibles en este contexto son casi ilimitadas. Tanto el acceso a los atributos como las llamadas a los métodos se realizarán de manera recurrente en diversas partes del código, con el fin de maximizar el rendimiento y aprovechar al máximo las capacidades de este tipo de bases de datos.

En el transcurso de la unidad, hemos revisado varios ejemplos prácticos que demuestran la necesidad de acceder a atributos y llamar a métodos, como en la ordenación de elementos y en la obtención de información sobre los objetos creados, entre otros.

/ 14. Bibliografía

Connolly, T. y Begg, C. (2005). *Sistemas de bases de datos* (5.ª ed.). Madrid, España. Addison-Wesley.

Elmasri, R. y Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison-Wesley.

López Montalbán, I. y De Castro, M. (2014). *Gestión de bases de datos: Técnico Superior en Administración de Sistemas Informáticos en Red*. Madrid. Garceta D.L.

Oppel, A. (2009). *Databases. A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España. Alfaomega.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos*. Paraninfo.

Vela Sánchez, B. (2015): *Diseño de bases de datos objeto-relacionales con UML*. Madrid, España. Dykinson.